Name: Phan Tran Thanh Huy

ID: ITCSIU22056

Lab 2

Part 1:

Exercise 1: UNIFORM DATA

Part A - Vertex Transformation

Assume a three-dimensional translation by <m, n, p>, identify each point (x, y, z) with the point (x, y, z, 1), the general form of this transformation in matrix form is:

$$\begin{bmatrix} 1 & 0 & 0 & m \\ 0 & 1 & 0 & n \\ 0 & 0 & 1 & p \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x+m \\ y+n \\ z+p \\ 1 \end{bmatrix}$$

The additional "1" in homogeneous coordinates allows translation to be represented as a matrix multiplication, so that all affine transformations (rotation, scaling, translation) can be combined into a single 4×4 transformation matrix.

Part B – Translation in Animation-1

1.

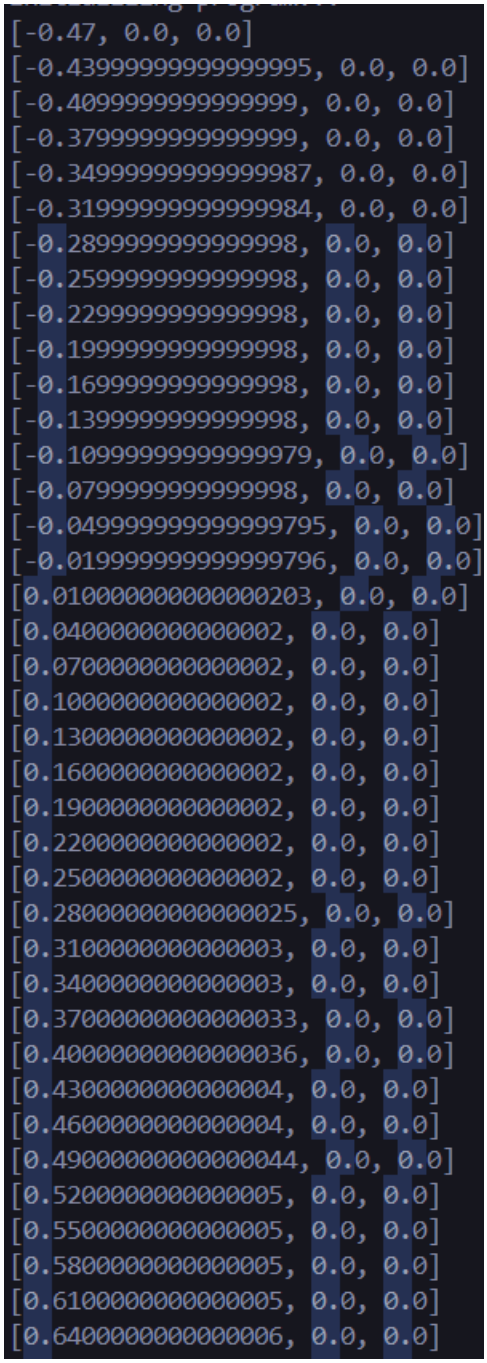
- We got the first point: (0.0, 0.2, 1). From translation= [-0.5, 0.0, 0.0]:

$$T = \begin{bmatrix} 1 & 0 & -0.5 \\ 0 & 1 & 0.0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Apply to the general form:

$$\begin{bmatrix} 1 & 0 & -0.5 \\ 0 & 1 & 0.0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 0.0 \\ 0.2 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.5 \\ 0.2 \\ 1 \end{bmatrix}$$

- The result will be



At each frame, the triangle will change the coordinate of x-axis by +0.03. So the triangle will move from left to right on the screen.

2. Why does the program reset the x-translation when x > 1.2?

In OpenGL clip space, the value of gl_Position (x, y, z) inside [-1, 1]

If x > 1, it will disappear from the screen.

⇨ x > 1.2 means the triangle will run completely out of the screen and appear on the left side of the screen.

Part C – Circular Motion in Animation-2Code:

1. The update rule is:

   x = r * cos(θ), y = r * sin(θ)

   with r = 0.75, θ = time

Derive these equations from the unit circle definition.

x(t) = 0.75cos(θ),

y(t) = 0.75sin(θ)

What is the translation matrix for a point (0,0,1) moved to (x,y) on the circle?

$$T = \begin{bmatrix} 1 & 0 & 0.75\cos(\theta) \\ 0 & 1 & 0.75\sin(\theta) \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.75\cos(\theta) \\ 0.75\sin(\theta) \\ 1 \end{bmatrix}$$

2. Show how the vertex (0.0, 0.2) is transformed at:

$$T = \begin{bmatrix} 1 & 0 & 0.75\cos(\theta) \\ 0 & 1 & 0.75\sin(\theta) \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 0.0 \\ 0.2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.75\cos(\theta) + 0.0 \\ 0.75\sin(\theta) + 0.2 \\ 1 \end{bmatrix}$$

- t = 0:

$$T = \begin{bmatrix} 0.75\cos(0) \\ 0.75\sin(0) \\ 1 \end{bmatrix} * \begin{bmatrix} 0 \\ 0.2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.75 \\ 0.2 \\ 1 \end{bmatrix}$$

- t = $\pi/2$

$$T = \begin{bmatrix} 0.75\cos(\pi/2) \\ 0.75\sin(\pi/2) \\ 1 \end{bmatrix} * \begin{bmatrix} 0 \\ 0.2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.75 \\ 0.95 \\ 1 \end{bmatrix}$$

- t = π:

$$T = \begin{bmatrix} 0.75\cos(\pi) \\ 0.75\sin(\pi) \\ 1 \end{bmatrix} * \begin{bmatrix} 0 \\ 0.2 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.75 \\ 0.2 \\ 1 \end{bmatrix}$$

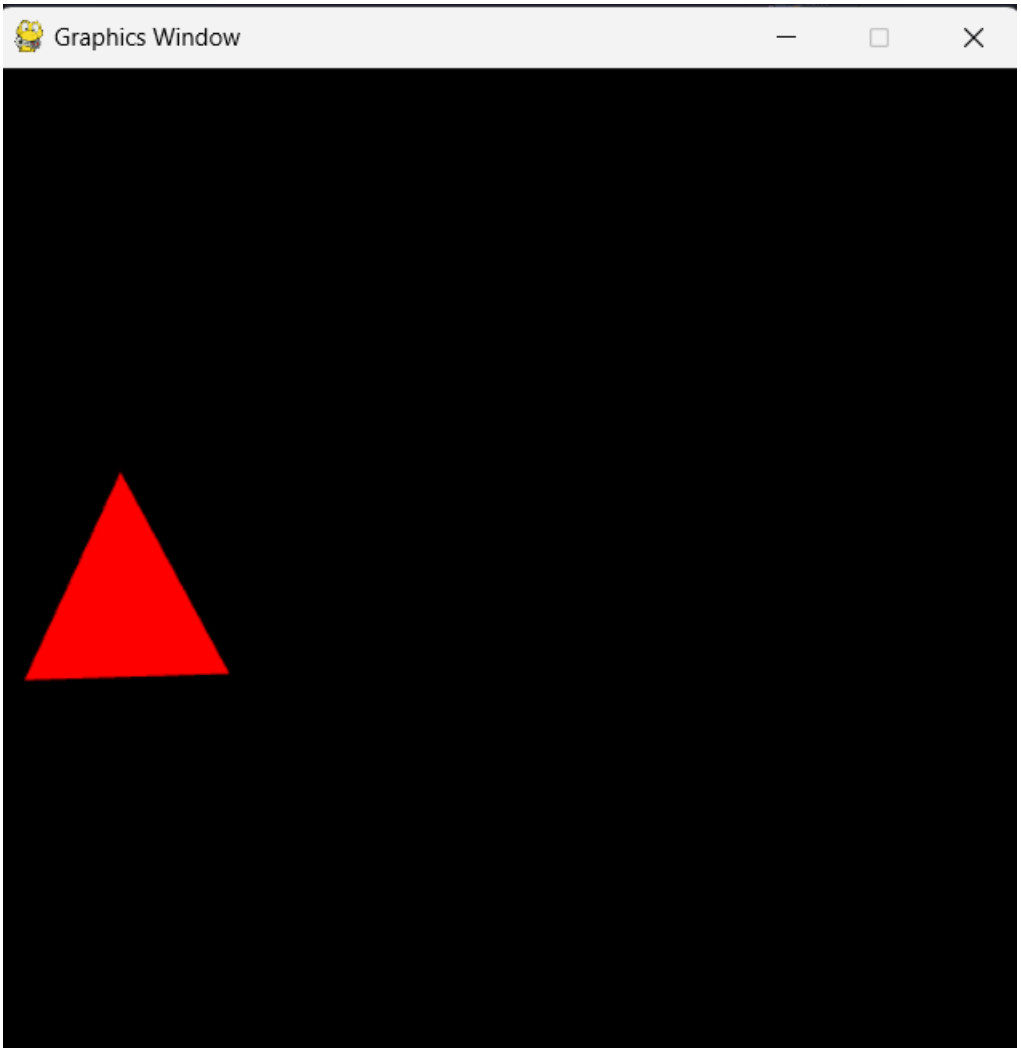
Part D – Implementation:

1. **Orbiting & Spinning Triangle**

Code:

```python
#!/usr/bin/python3
import math

import OpenGL.GL as GL

from py3d.core.base import Base
from py3d.core.utils import Utils
from py3d.core.attribute import Attribute
from py3d.core.uniform import Uniform


class Example(Base):
    """ Animate a triangle moving in a circular path around the origin """
    def initialize(self):
        print("Initializing program...")
        # Initialize program #
        vs_code = """
            in vec3 position;
            uniform vec3 translation;
            uniform vec3 center;
            uniform float angle;
            void main()
            {
                float c = cos(angle);
                float s = sin(angle);
                mat2 rotation = mat2(c, -s, s, c);

                vec2 local = position.xy - center.xy;
                vec2 rotated = rotation * local;
                vec2 world = rotated + center.xy + translation.xy;

                gl_Position = vec4(world.x, world.y, position.z + translation.z, 1.0);
            }
        """
        fs_code = """
            uniform vec3 baseColor;
            out vec4 fragColor;
            void main()
            {
                fragColor = vec4(baseColor.r, baseColor.g, baseColor.b, 1.0);
            }
        """
        self.program_ref = Utils.initialize_program(vs_code, fs_code)
        # Render settings (optional) #
        # Specify color used when clearly
        GL.glClearColor(0.0, 0.0, 0.0, 1.0)
        # Set up vertex array object #
        vao_ref = GL.glGenVertexArrays(1)
        GL.glBindVertexArray(vao_ref)
        # Set up vertex attribute #
        self.position_data = [[ 0.0,  0.2,  0.0],
                              [ 0.2, -0.2,  0.0],
                              [-0.2, -0.2,  0.0]]
        self.vertex_count = len(self.position_data)
        position_attribute = Attribute('vec3', self.position_data)
        position_attribute.associate_variable(self.program_ref, 'position')

        cx = sum(v[0] for v in self.position_data) / self.vertex_count
        cy = sum(v[1] for v in self.position_data) / self.vertex_count
        cz = 0.0
        # Set up uniforms #
        self.translation = Uniform('vec3', [0.0, 0.0, 0.0])
        self.translation.locate_variable(self.program_ref, 'translation')

        self.center = Uniform('vec3', [cx, cy, cz])
        self.center.locate_variable(self.program_ref, 'center')

        self.angle = Uniform('float', 0.0)
        self.angle.locate_variable(self.program_ref, 'angle')

        self.base_color = Uniform('vec3', [1.0, 0.0, 0.0])
        self.base_color.locate_variable(self.program_ref, 'baseColor')

        self.orbit_radius = 0.75
        self.orbit_speed = 1.0   # radians per second
        self.spin_speed = 2.0

    def update(self):
        """ Update data """
        t = self.time
        self.translation.data[0] = 0.75 * math.cos(self.time)
        self.translation.data[1] = 0.75 * math.sin(self.time)
        self.angle.data = self.spin_speed * t
        # Reset color buffer with specified color
        GL.glClear(GL.GL_COLOR_BUFFER_BIT)
        GL.glUseProgram(self.program_ref)
        self.translation.upload_data()
        self.center.upload_data()
        self.angle.upload_data()
        self.base_color.upload_data()
        GL.glDrawArrays(GL.GL_TRIANGLES, 0, self.vertex_count)


# Instantiate this class and run the program
Example().run()
```

Result:

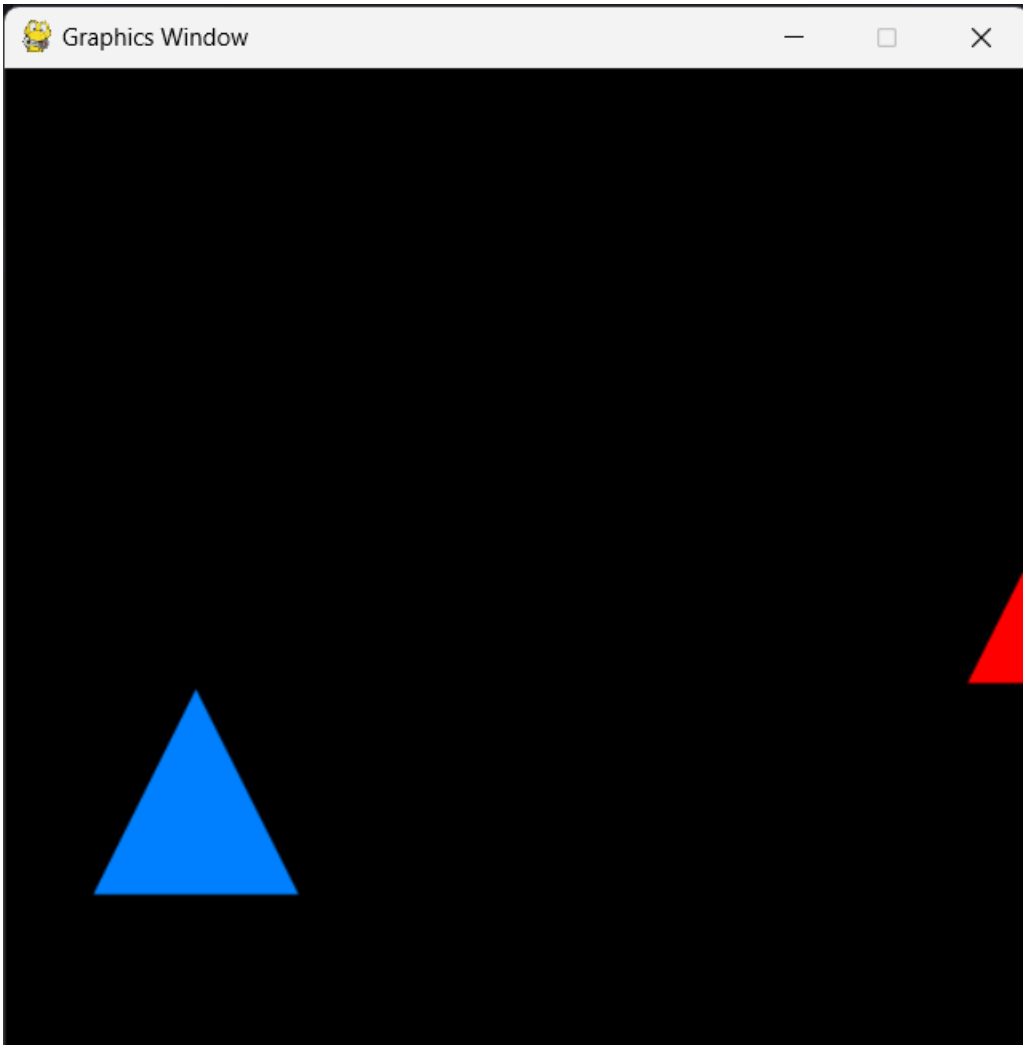


2. Two Triangles (Linear vs Circular Motion)

Code:

```python
#!/usr/bin/python3
import OpenGL.GL as GL
import math

from py3d.core.base import Base
from py3d.core.utils import Utils
from py3d.core.attribute import Attribute
from py3d.core.uniform import Uniform


class Example(Base):
    """ Animate a triangle moving across screen """
    def initialize(self):
        print("Initializing program...")
        # Initialize program #
        vs_code = """
            in vec3 position;
            uniform vec3 translation;
            void main()
            {
                vec3 pos = position + translation;
                gl_Position = vec4(pos.x, pos.y, pos.z, 1.0);
            }
        """
        fs_code = """
            uniform vec3 baseColor;
            out vec4 fragColor;
            void main()
            {
                fragColor = vec4(baseColor.r, baseColor.g, baseColor.b, 1.0);
            }
        """
        self.program_ref = Utils.initialize_program(vs_code, fs_code)
        # render settings (optional) #
        # Specify color used when clearly
        GL.glClearColor(0.0, 0.0, 0.0, 1.0)
        # Set up vertex array object #
        vao_ref = GL.glGenVertexArrays(1)
        GL.glBindVertexArray(vao_ref)
        # Set up vertex attribute #
        position_data = [[ 0.0,  0.2,  1],
                         [ 0.2, -0.2,  0.0],
                         [-0.2, -0.2,  0.0]]
        self.vertex_count = len(position_data)
        position_attribute = Attribute('vec3', position_data)
        position_attribute.associate_variable(self.program_ref, 'position')
        # Set up uniforms #
        self.translation = Uniform('vec3', [-0.5, 0.0, 0.0])
        self.translation.locate_variable(self.program_ref, 'translation')
        self.base_color = Uniform('vec3', [1.0, 0.0, 0.0])
        self.base_color.locate_variable(self.program_ref, 'baseColor')

        self.linear_pos = -1
        self.orbit_radius = 0.75

    def update(self):
        """ Update data """
        t = self.time
        GL.glClear(GL.GL_COLOR_BUFFER_BIT)
        GL.glUseProgram(self.program_ref)

        # --- Triangle 1: moves linearly across screen ---
        self.linear_pos += 0.02
        if self.linear_pos > 1.2:
            self.linear_pos = -1.2

        self.translation.data = [self.linear_pos, 0.0, 0.0]
        self.base_color.data = [1.0, 0.0, 0.0]  # Red
        self.translation.upload_data()
        self.base_color.upload_data()
        GL.glDrawArrays(GL.GL_TRIANGLES, 0, self.vertex_count)

        # --- Triangle 2: orbits around origin ---
        tx = self.orbit_radius * math.cos(t)
        ty = self.orbit_radius * math.sin(t)
        self.translation.data = [tx, ty, 0.0]
        self.base_color.data = [0.0, 0.5, 1.0]  # Blue
        self.translation.upload_data()
        self.base_color.upload_data()
        GL.glDrawArrays(GL.GL_TRIANGLES, 0, self.vertex_count)


# Instantiate this class and run the program
Example().run()
```

Result:

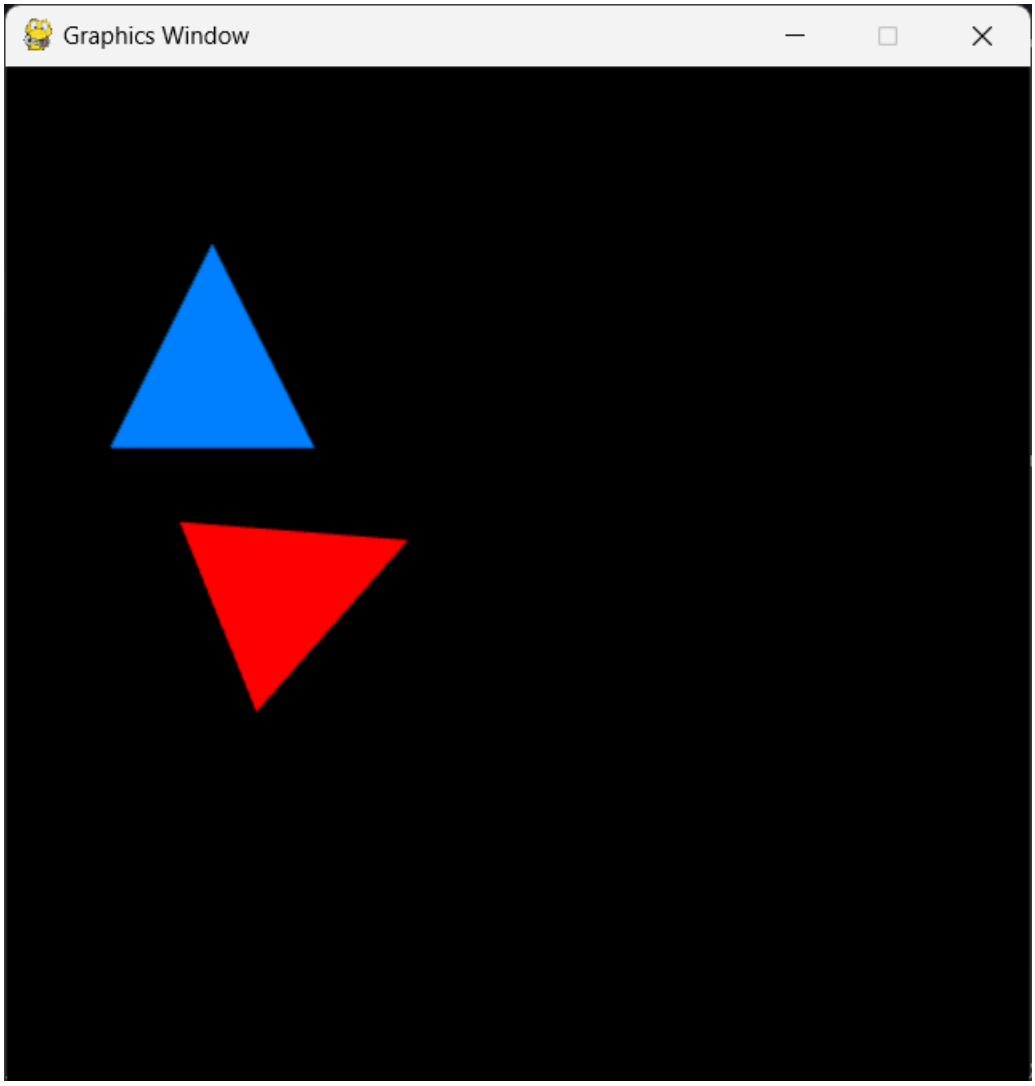


3. Linear motion with Rotation + Circular Motion

Code:

```python
#!/usr/bin/python3
import OpenGL.GL as GL
import math

from py3d.core.base import Base
from py3d.core.utils import Utils
from py3d.core.attribute import Attribute
from py3d.core.uniform import Uniform


class Example(Base):
    """ Animate a triangle moving across screen """
    def initialize(self):
        print("Initializing program...")
        # Initialize program #
        vs_code = """
            in vec3 position;
            uniform vec3 translation;
            uniform float angle;
            void main()
            {
                // rotation matrix
                float c = cos(angle);
                float s = sin(angle);
                mat2 R = mat2(c, -s, s, c);

                // apply rotation + translation
                vec2 rotated = R * position.xy;
                vec2 world = rotated + translation.xy;

                gl_Position = vec4(world, position.z, 1.0);
            }
        """
        fs_code = """
            uniform vec3 baseColor;
            out vec4 fragColor;
            void main()
            {
                fragColor = vec4(baseColor.r, baseColor.g, baseColor.b, 1.0);
            }
        """
        self.program_ref = Utils.initialize_program(vs_code, fs_code)
        # render settings (optional) #
        # Specify color used when clearly
        GL.glClearColor(0.0, 0.0, 0.0, 1.0)
        # Set up vertex array object #
        vao_ref = GL.glGenVertexArrays(1)
        GL.glBindVertexArray(vao_ref)
        # Set up vertex attribute #
        position_data = [[ 0.0,  0.2,  1],
                         [ 0.2, -0.2,  0],
                         [-0.2, -0.2,  0.0]]
        self.vertex_count = len(position_data)
        position_attribute = Attribute('vec3', position_data)
        position_attribute.associate_variable(self.program_ref, 'position')
        # Set up uniforms #
        self.translation = Uniform('vec3', [-0.5, 0.0, 0.0])
        self.translation.locate_variable(self.program_ref, 'translation')
        self.base_color = Uniform('vec3', [1.0, 0.0, 0.0])
        self.base_color.locate_variable(self.program_ref, 'baseColor')
        self.angle = Uniform('float', 0.0)
        self.angle.locate_variable(self.program_ref, 'angle')

        self.linear_pos = -1
        self.orbit_radius = 0.75

    def update(self):
        """ Update data """
        t = self.time
        GL.glClear(GL.GL_COLOR_BUFFER_BIT)
        GL.glUseProgram(self.program_ref)

        # --- Triangle 1: moves linearly across screen ---
        self.linear_pos += 0.02
        if self.linear_pos > 1.2:
            self.linear_pos = -1.2

        self.translation.data = [self.linear_pos, 0.0, 0.0]
        self.angle.data = t * 3.0
        self.base_color.data = [1.0, 0.0, 0.0]  # Red
        self.translation.upload_data()
        self.angle.upload_data()
        self.base_color.upload_data()
        GL.glDrawArrays(GL.GL_TRIANGLES, 0, self.vertex_count)

        # --- Triangle 2: orbits around origin ---
        tx = self.orbit_radius * math.cos(t)
        ty = self.orbit_radius * math.sin(t)
        self.translation.data = [tx, ty, 0.0]
        self.angle.data = 0.0
        self.base_color.data = [0.0, 0.5, 1.0]  # Blue
        self.translation.upload_data()
        self.angle.upload_data()
        self.base_color.upload_data()
        GL.glDrawArrays(GL.GL_TRIANGLES, 0, self.vertex_count)


# Instantiate this class and run the program
Example().run()
```

Result:



Exercise 2: Matrix Algebra and Transformations

Part A – Adding Keyboard Control

```python
#!/usr/bin/python3
from math import pi
import OpenGL.GL as GL

from py3d.core.base import Base
from py3d.core_ext.camera import Camera
from py3d.core_ext.mesh import Mesh
from py3d.core_ext.renderer import Renderer
from py3d.core_ext.scene import Scene
from py3d.geometry.sphere import SphereGeometry
from py3d.material.material import Material
from py3d.core.matrix import Matrix


class Example(Base):
    """ Render a spinning sphere with gradient colors """

    def initialize(self):
        print("Initializing program...")
        # --- Setup basic 3D scene ---
        self.renderer = Renderer()
        self.scene = Scene()
        self.camera = Camera(aspect_ratio=800 / 600)
        self.camera.set_position([0, 0, 7])

        # --- Sphere mesh ---
        geometry = SphereGeometry(radius=2)
        vs_code = """
        uniform mat4 modelMatrix;
        uniform mat4 viewMatrix;
        uniform mat4 projectionMatrix;
        in vec3 vertexPosition;
        out vec3 position;
        void main()
        {
            vec4 pos = vec4(vertexPosition, 1.0);
            gl_Position = projectionMatrix * viewMatrix * modelMatrix * pos;
            position = vertexPosition;
        }
        """
        fs_code = """
        in vec3 position;
        out vec4 fragColor;
        void main()
        {
            vec3 color = mod(position, 1.0);
            fragColor = vec4(color, 1.0);
        }
        """
        material = Material(vs_code, fs_code)
        material.locate_uniforms()
        self.mesh = Mesh(geometry, material)
        self.scene.add(self.mesh)

        # --- Movement parameters ---
        self.move_speed = 2.0  # units per second
        self.turn_speed = 90 * (pi / 180)  # radians per second

    def update(self):
        """Update transformations per frame."""
        move_amount = self.move_speed * self.delta_time
        turn_amount = self.turn_speed * self.delta_time

        # --- Auto-spin (always active) ---
        self.mesh.rotate_y(0.00514)
        self.mesh.rotate_x(0.00337)

        if self.input.is_key_pressed('w'):
            self.mesh.local_matrix = Matrix.make_translation(0, move_amount, 0) @ self.mesh.local_matrix
        if self.input.is_key_pressed('s'):
            self.mesh.local_matrix = Matrix.make_translation(0, -move_amount, 0) @ self.mesh.local_matrix
        if self.input.is_key_pressed('a'):
            self.mesh.local_matrix = Matrix.make_translation(-move_amount, 0, 0) @ self.mesh.local_matrix
        if self.input.is_key_pressed('d'):
            self.mesh.local_matrix = Matrix.make_translation(move_amount, 0, 0) @ self.mesh.local_matrix
        if self.input.is_key_pressed('z'):
            self.mesh.local_matrix = Matrix.make_translation(0, 0, move_amount) @ self.mesh.local_matrix
        if self.input.is_key_pressed('x'):
            self.mesh.local_matrix = Matrix.make_translation(0, 0, -move_amount) @ self.mesh.local_matrix

        if self.input.is_key_pressed('i'):
            self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_translation(0, move_amount, 0)
        if self.input.is_key_pressed('k'):
            self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_translation(0, -move_amount, 0)
        if self.input.is_key_pressed('j'):
            self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_translation(-move_amount, 0, 0)
        if self.input.is_key_pressed('l'):
            self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_translation(move_amount, 0, 0)

        # --- Render frame ---
        self.renderer.render(self.scene, self.camera)


# Run program
Example(screen_size=[800, 600]).run()
```

- Explain why global movement uses: M = T * M, while local movement uses: M = M * T.

➔ Global movement uses M = T * M because the transformation T is applied in world coordinates before the object's existing transform, so the object moves relative to the world axes.

➔ Local movement uses M = M * T because the transformation T is applied in the object's local coordinates, after its current transform, so the object moves relative to its own axes.

Part B – Interactive Rotations

```
89
90          if self.input.is_key_pressed('q'):
91              self.mesh.local_matrix = Matrix.make_rotation_y(turn_amount) @ self.mesh.local_matrix
92          if self.input.is_key_pressed('e'):
93              self.mesh.local_matrix = Matrix.make_rotation_y(-turn_amount) @ self.mesh.local_matrix
94
95          if self.input.is_key_pressed('u'):
96              self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_rotation_y(turn_amount)
97          if self.input.is_key_pressed('o'):
98              self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_rotation_y(-turn_amount)
99
```

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Explain the difference between rotating around the world axis and rotating around the object's local axis.

➔ Rotating around the world axis means the object rotates with respect to the fixed world coordinate system. Its rotation does not depend on the object's current orientation. (Matrix form: M = R × M).

➔ Rotating around the object's local axis means the object rotates with respect to its own coordinate system, which moves and turns together with the object. (Matrix form: M = M × R).

Part C – Auto-Spin and Combination

```python
#!/usr/bin/python3
from math import pi
import OpenGL.GL as GL

from py3d.core.base import Base
from py3d.core_ext.camera import Camera
from py3d.core_ext.mesh import Mesh
from py3d.core_ext.renderer import Renderer
from py3d.core_ext.scene import Scene
from py3d.geometry.sphere import SphereGeometry
from py3d.material.material import Material
from py3d.core.matrix import Matrix


class Example(Base):
    """ Render a spinning sphere with gradient colors """

    def initialize(self):
        print("Initializing program...")
        # --- Setup basic 3D scene ---
        self.renderer = Renderer()
        self.scene = Scene()
        self.camera = Camera(aspect_ratio=800 / 600)
        self.camera.set_position([0, 0, 7])

        # --- Sphere mesh ---
        geometry = SphereGeometry(radius=2)
        vs_code = """
        uniform mat4 modelMatrix;
        uniform mat4 viewMatrix;
        uniform mat4 projectionMatrix;
        in vec3 vertexPosition;
        out vec3 position;
        void main()
        {
            vec4 pos = vec4(vertexPosition, 1.0);
            gl_Position = projectionMatrix * viewMatrix * modelMatrix * pos;
            position = vertexPosition;
        }
        """
        fs_code = """
        in vec3 position;
        out vec4 fragColor;
        void main()
        {
            vec3 color = mod(position, 1.0);
            fragColor = vec4(color, 1.0);
        }
        """
        material = Material(vs_code, fs_code)
        material.locate_uniforms()
        self.mesh = Mesh(geometry, material)
        self.scene.add(self.mesh)

        # --- Movement parameters ---
        self.move_speed = 2.0  # units per second
        self.turn_speed = 90 * (pi / 180)  # radians per second

    def update(self):
        """Update transformations per frame."""
        move_amount = self.move_speed * self.delta_time
        turn_amount = self.turn_speed * self.delta_time

        # --- Auto-spin (always active) ---
        self.mesh.rotate_y(0.00514)
        self.mesh.rotate_x(0.00337)

        if self.input.is_key_pressed('w'):
            self.mesh.local_matrix = Matrix.make_translation(0, move_amount, 0) @ self.mesh.local_matrix
        if self.input.is_key_pressed('s'):
            self.mesh.local_matrix = Matrix.make_translation(0, -move_amount, 0) @ self.mesh.local_matrix
        if self.input.is_key_pressed('a'):
            self.mesh.local_matrix = Matrix.make_translation(-move_amount, 0, 0) @ self.mesh.local_matrix
        if self.input.is_key_pressed('d'):
            self.mesh.local_matrix = Matrix.make_translation(move_amount, 0, 0) @ self.mesh.local_matrix
        if self.input.is_key_pressed('z'):
            self.mesh.local_matrix = Matrix.make_translation(0, 0, move_amount) @ self.mesh.local_matrix
        if self.input.is_key_pressed('x'):
            self.mesh.local_matrix = Matrix.make_translation(0, 0, -move_amount) @ self.mesh.local_matrix

        if self.input.is_key_pressed('i'):
            self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_translation(0, move_amount, 0)
        if self.input.is_key_pressed('k'):
            self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_translation(0, -move_amount, 0)
        if self.input.is_key_pressed('j'):
            self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_translation(-move_amount, 0, 0)
        if self.input.is_key_pressed('l'):
            self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_translation(move_amount, 0, 0)

        if self.input.is_key_pressed('q'):
            self.mesh.local_matrix = Matrix.make_rotation_y(turn_amount) @ self.mesh.local_matrix
        if self.input.is_key_pressed('e'):
            self.mesh.local_matrix = Matrix.make_rotation_y(-turn_amount) @ self.mesh.local_matrix

        if self.input.is_key_pressed('u'):
            self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_rotation_y(turn_amount)
        if self.input.is_key_pressed('o'):
            self.mesh.local_matrix = self.mesh.local_matrix @ Matrix.make_rotation_y(-turn_amount)

        # --- Render frame ---
        self.renderer.render(self.scene, self.camera)



# Run program
Example(screen_size=[800, 600]).run()

```

Part 2: Homework

Question 1: Spinning Sun

1.

```
geometry = SphereGeometry(radius=1)
```

2.

```
void main()
{
    vec3 color = mod(position, 1.0);
    fragColor = vec4(color, 1.0);
}
```

3.

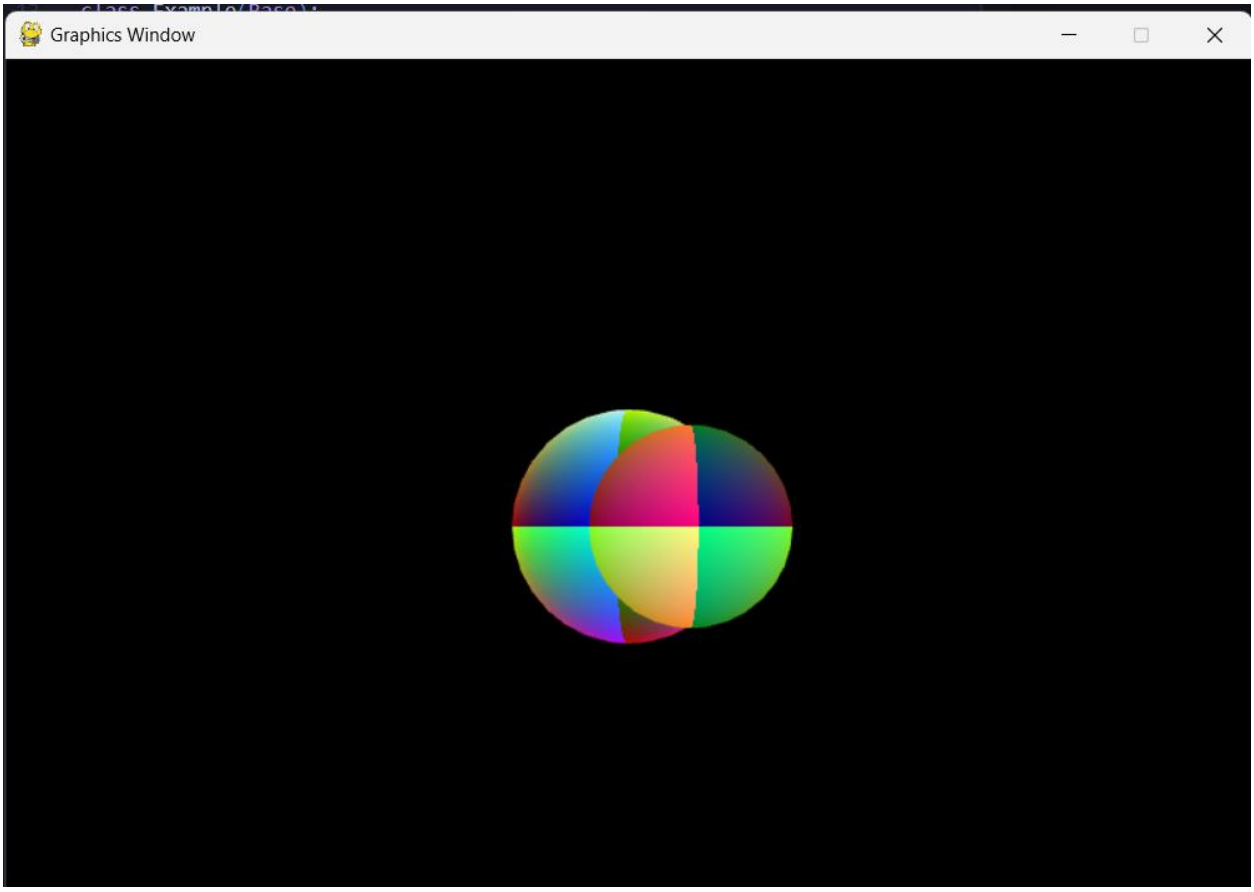
```
def update(self):
    self.mesh.rotate_y(0.01)
    self.renderer.render(self.scene, self.camera)
```

Question 2: Adding the Earth (Orbit Only)

Code:

```python
#!/usr/bin/python3
from math import pi, sin, cos
from py3d.core.base import Base
from py3d.core_ext.camera import Camera
from py3d.core_ext.mesh import Mesh
from py3d.core_ext.renderer import Renderer
from py3d.core_ext.scene import Scene
from py3d.geometry.sphere import SphereGeometry
from py3d.material.material import Material
from py3d.core.matrix import Matrix


class Example(Base):
    """ Render a spinning sphere with gradient colors """
    def initialize(self):
        print("Initializing program...")
        self.renderer = Renderer()
        self.scene = Scene()
        self.camera = Camera(aspect_ratio=800/600)
        self.camera.set_position([0, 0, 7])
        vs_code = """
        uniform mat4 modelMatrix;
        uniform mat4 viewMatrix;
        uniform mat4 projectionMatrix;
        in vec3 vertexPosition;
        out vec3 position;
        void main()
        {
            vec4 pos = vec4(vertexPosition, 1.0);
            gl_Position = projectionMatrix * viewMatrix * modelMatrix * pos;
            position = vertexPosition;
        }
        """
        fs_code = """
        in vec3 position;
        out vec4 fragColor;
        void main()
        {
            vec3 color = mod(position, 1.0);
            fragColor = vec4(color, 1.0);
        }
        """
        material = Material(vs_code, fs_code)
        material.locate_uniforms()

        sun_geometry = SphereGeometry(radius=1.0)
        self.sun = Mesh(sun_geometry, material)
        self.scene.add(self.sun)

        earth_geometry = SphereGeometry(radius=0.5)
        self.earth = Mesh(earth_geometry, material)
        self.scene.add(self.earth)

        self.orbit_radius = 3.0
        self.orbit_angle = 0.0

    def update(self):
        self.sun.rotate_y(0.01)

        self.orbit_angle += 0.01
        x = self.orbit_radius * cos(self.orbit_angle)
        z = self.orbit_radius * sin(self.orbit_angle)
        self.earth.set_position([x, 0, z])

        self.renderer.render(self.scene, self.camera)


# Instantiate this class and run the program
Example(screen_size=[800, 600]).run()
```

Result:



Question 3: Earth's Self-Spin

1.

```
57      def update(self):
58          self.sun.rotate_y(0.01)
59
60          self.orbit_angle += 0.01
61          x = self.orbit_radius * cos(self.orbit_angle)
62          z = self.orbit_radius * sin(self.orbit_angle)
63          self.earth.set_position([x, 0, z])
64
65          self.earth.rotate_y(0.05)
66
67          self.renderer.render(self.scene, self.camera)
68
```

2. The transformation equation for Earth's model matrix is:

$$M_{earth} = T_{orbit} \times R_{self}$$

Where:

$$T_{orbit} = \begin{bmatrix} 1 & 0 & 0 & x \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_{self} = R_y(\theta_{self})$$

3. Explain why both transformations are needed to simulate realistic Earth motion:

    - Orbit translation moves the Earth around the Sun — this represents its revolution (the Earth traveling around the Sun once a year).
    - Self rotation makes the Earth spin around its own axis — this represents its day/night cycle (the Earth rotating once every 24 hours).

➔ Without self-rotation, Earth would orbit the Sun but always show the same side (like the Moon).
➔ Without orbit translation, Earth would spin in place but not revolve around the Sun.

Question 4: Adding the Moon

1.

```
moon_geometry = SphereGeometry(radius=0.2)
self.moon = Mesh(moon_geometry, material)
self.scene.add(self.moon)

self.orbit_radius = 3.0
self.moon_orbit_radius = 1.0
self.orbit_angle = 0.0
self.moon_orbit_angle = 0.0
```

2.

```
        self.moon_orbit_angle += 0.05
        x_moon = x_earth + self.moon_orbit_radius * cos(self.moon_orbit_angle)
        z_moon = z_earth + self.moon_orbit_radius * sin(self.moon_orbit_angle)
        self.moon.set_position([x_moon, 0, z_moon])
```

3.

```
        self.moon.rotate_y(0.1)
```

Result: